



Sonthillu Constructions — Associate Partner Portal

Master Project Blueprint & Antigravity Build Specification

React.js + Node.js | Standalone Web App + PWA | Hostinger Deployment | Version 2.0 — August 2026

Single source of truth for rebuilding the standalone Associate Partner Portal in Antigravity with React.js + Node.js, PWA support and Hostinger deployment.

1. Executive Definition

Build a completely standalone Associate Partner Portal for Sonthillu Constructions. It is one responsive web application that works in a normal desktop/mobile browser and can also be installed as a PWA. The frontend must be React.js and the backend Node.js. The portal owns its own authentication, authorization, database, workflows, records and UI. It must not connect to the CRM, EMS, Customer Portal, existing CRM/EMS databases, or unrequested external business APIs.

- Primary roles: MD, Associate Manager (AM), Associate.
- Desktop: persistent sidebar + top utility header. Mobile: compact header + bottom navigation for core functions + drawer/hamburger for secondary functions.
- PWA: manifest, icons, standalone display mode, service worker/installability; never fake an Installed state.
- Hostinger target: deploy the React production build as the web frontend and the Node.js API through the hosting environment's supported Node application/reverse-proxy configuration. Keep frontend and API base URLs configurable.
- Use the supplied Sonthillu logo and the supplied Master UI Design System as the visual source of truth.

2. Source-of-Truth Documents

- Associate Partner Portal — Master Development Instructions: defines independence, roles, security, phase verification, data integrity and completion rules.
- associate portal app.pdf: defines portal information architecture, workflows, fields, modules, approval matrix, dashboards, carousel, popup, reviews, PWA, hierarchy and recommended phases.
- Sonthillu Constructions — Master UI Design System: defines visual language, palette, typography, spacing, components, responsive behavior and QA standards.
- Supplied exact Sonthillu logo SVG: use as the brand asset; do not redraw or substitute it.

3. Non-Negotiable Rules

- Do not integrate CRM, EMS, Customer Portal or existing business systems.
- Do not invent major business functionality that is not in this specification.
- Do not hide security behind frontend route guards. Authorization must be enforced on the server and data-access layer.
- Do not use any, TypeScript suppression, disabled lint rules, deleted tests, commented-out broken functionality, swallowed exceptions or fake success states to make a build pass.
- Important historical records must be traceable. Prefer Active/Inactive/Pending/Approved/Rejected/Cancelled/Archived statuses and soft-delete behavior.
- After every phase: build, type-check, lint, tests, runtime/console checks, routes, responsive behavior and relevant manual workflows must pass before moving on.
- Never proceed to the next phase with known blocking errors.

4. Brand & UI Design System

The UI must feel like a premium real-estate SaaS product: calm, airy, trustworthy, structured and modern. The source design system specifies pale blue surfaces, architectural navy structure, warm gold identity, restrained action blue, rounded modular cards and precise spacing. Gold is an accent, not a full-page background.

- Primary Navy: #203B73
- Brand Gold: #E0B040
- App Background: #F4FAFC
- Deep Navy: #172A52
- Action Blue: #4268E8
- Card Surface: #EAF3F8
- Primary Text: #172033
- Muted Text: #667085
- Border: #D7E3EA
- Typography: Inter, Manrope, Plus Jakarta Sans or equivalent; keep hierarchy consistent.

- Display: 32–40px/700; page title 24–28px/700; section 16–20px/650; body 14–16px; caption 12–13px.
- Spacing: 4px base; preferred rhythm 8/12/16/20/24/32/40px.
- Cards: 14–20px radius, 20–24px padding for primary cards; inputs/controls 10–12px radius and minimum 44px height.
- Desktop: 12-column grid, 20–24px gutters, ~240–260px sidebar.
- Tablet: 8-column layout, collapsed sidebar.
- Mobile: 4-column layout, drawer/bottom navigation, stacked cards, horizontal filter scrolling and card-based tables.
- Use one professional SVG icon library consistently; do not use emoji as primary navigation icons.
- Every major component needs normal, hover, focus, loading, empty, success and error states where applicable.
- Use restrained 150–250ms micro-interactions. Avoid excessive gradients, blur, glow, shadows and animation.

5. Application Shell

- Desktop navigation: Dashboard, Projects, Bookings, Commissions, Team, Offers, Pending Requests, Travel Allowance, Site Visits, Notifications, Reviews, My Profile, FAQ, Tutorials.
- Top bar: global search, quick actions where applicable, notifications and profile/account menu.
- Mobile: core bottom navigation plus hamburger/drawer for secondary sections.
- Use a single responsive codebase; do not create a separate mobile application.
- Prevent layout overflow at every breakpoint. Main content must never be covered by a desktop sidebar on mobile.
- Sidebar/drawer state must be controlled by responsive CSS/layout state, not by fragile fixed offsets.
- Use safe-area insets on mobile where needed.

6. Authentication & Session Security

- Use secure password hashing on the backend; never store plaintext passwords.
- JWT access authentication may be implemented with secure HttpOnly cookies. If frontend/API are cross-site, configure CORS and cookie attributes deliberately and verify them in production rather than guessing.
- MD and AM are fixed/pre-created accounts with temporary initial passwords.
- First successful login forces password change; temporary password becomes invalid after change.
- MD and AM support exactly one active device/session. A second login is rejected with a clear message. Profile can show active device/session information and allow revoke/sign-out.
- Associates may use normal multi-device access unless changed later.
- Backend authorization must check role, ownership and hierarchy on every protected data access.
- Do not trust role, user ID, parent ID or visibility scope supplied by the browser.
- Sensitive fields (Aadhaar, PAN, bank account, IFSC, emergency contact, private address) require server-side authorization and masking where displayed.
- Add rate limiting/lockout controls for login, secure logout, session revocation, validation, CSRF protection where cookie-auth architecture requires it, and security headers.
- Log security-relevant events without logging passwords, tokens or unnecessary sensitive data.

7. Associate Hierarchy & Visibility

The team is a downward-only hierarchy. If A is above B and C, and B is above D and E, A can see A/B/C/D/E; B can see B/D/E; B cannot see A/C. The same rule applies to team bookings and other hierarchy-scoped data. This must be enforced by backend query/data-access rules.

- Store an explicit parent/team relationship and prevent cycles.
- Provide list view and tree view.
- Associate sees only allowed team fields: name, code, commission, status, team count.
- Team member phone, email, Aadhaar, PAN, bank, address and emergency details are hidden.
- MD can see the complete hierarchy; AM can manage operational data according to the approval matrix.

8. Role Capability Summary

- MD: highest authority; all associates; all team hierarchy; final approvals; project verification; commission policy authority; travel approval; critical request approval; sensitive data access as permitted; analytics and audit trail.
- AM: operational controller; associates; projects; carousel; offers; onboarding; team requests; booking verification; commission configuration; reviews; portal content; cannot override MD-controlled approvals.
- Associate: sales/partner user; dashboard; assigned projects; bookings; own/downline bookings according to hierarchy; commission; team; offers; travel requests; site visits; review requests; notifications; profile; FAQ/tutorials; referrals.

9. MD Dashboard & Controls

- KPI cards: Associates, Teams, Bookings, Commission.
- Pending approvals: new associates, travel requests, project approvals and other critical requests.
- Associate directory: code, name, manager, team, status, commission, bookings, reviews, performance.
- Team tree view.
- Central Pending Approvals queue with request type, requester, date, reason, supporting information, status, approve/reject and mandatory rejection reason.
- Performance leaderboard with filters Today/Month/Quarter/Year/Project and metrics bookings, site visits, conversion, commission, reviews and team performance.
- Optional associate levels: Starter, Bronze, Silver, Gold, Platinum, Elite. If implemented, keep calculation configurable and avoid exposing sensitive information.
- Optional leaderboard visibility toggle controlled by MD/AM.

10. Associate Manager Operations

- Create and modify associates; system-generated permanent associate code.
- Review associate referrals and prepare new associates for MD approval.
- Manage projects, project media, offers, carousel, tutorials and FAQ.
- Verify bookings.
- Configure commission type/value, subject to MD final authority where policy changes require approval.
- Manage team requests and operational content.
- Never manually create a 'MD Verified' project badge; that badge appears only after MD approval.

11. Associate Dashboard

- Personalized greeting and associate code/level.
- KPI cards: My Bookings, Commission, Team, Site Visits.
- Featured Projects.
- Active Offers.
- Upcoming Site Visits.
- Primary quick actions should make Booking, Site Visit, Review Request and Travel Request easy to reach on mobile.
- Global search must respect permissions.

12. Projects

- Project card: name, verified badge, location, type, status, brochure, layout, gallery, available inventory count, project details, associate offer, customer offer.
- Project detail tabs: Overview, Gallery, Layout, Brochure, Available Units, Offers.
- Only available inventory is shown to associates.
- Inventory examples should distinguish AVAILABLE from BOOKED/other statuses; booked units must disappear from associate-available inventory after verified booking.
- Project lifecycle: draft → submitted/pending approval → approved/verified or rejected → active/inactive/archived as applicable.

- MD approval triggers the Verified Project badge.

13. Booking System

- Associate can create booking with customer details: name, phone, alternate phone, email, address, preferred location.
- Property: project, plot/unit, type, size, price.
- Booking: booking date, expected amount, payment mode, booking amount, notes, documents.
- Workflow: SUBMITTED → AM verification/UNDER REVIEW → VERIFIED → Booking Confirmed; support REJECTED, PAYMENT PENDING and CANCELLED where applicable.
- Prevent booking of unavailable inventory at server level, not just UI level.
- After verified booking, inventory must no longer appear as available.
- Associate sees My Bookings and permitted My Team Bookings using downward hierarchy.
- Add duplicate/conflict checks for the same unit and appropriate booking/customer validation.

14. Commission

- Dashboard: Total Commission, Received, Pending.
- Ledger columns: Date, Booking, Project, Commission, Received, Due, Status.
- Filters: Today, This Week, This Month, Custom Date, Year, Project.
- Commission type: Percentage or Fixed Amount.
- Financial changes require audit logging; policy changes should follow MD final authority.

15. Team

- List view and tree view.
- Downline-only visibility for associates.
- Team count and statistics.
- Add/remove team member requests follow approval workflow.
- Never expose sensitive personal/banking data through team endpoints.

16. Travel Allowance

- Request form: travel date, from, to, purpose, project, customer, distance, travel mode, amount requested, supporting bill, notes.
- Workflow: Pending → MD Review → Approved/Rejected → Paid.
- Associate sees own travel records.
- Rejection reason and payment updates are stored historically.

17. Site Visits

- Sections: My Site Visits and Team Site Visits.
- Fields: customer, project, date, time, associate, status, outcome.
- Statuses: Scheduled, On the Way, Arrived, Customer Met, Completed, Cancelled, Rescheduled.
- Create/update/outcome forms include customer details, number of visitors, purpose, follow-up requirement/date and remarks.

18. Offers & Achievement

- Separate Associate Offers, Customer Offers, Project Offers and Achieved Offers.
- Achievement UI can show progress, reward and remaining bookings.
- Offer lifecycle: draft/active/inactive/expired/archived as appropriate.
- First-visit primary offer popup is controlled by AM: promotional image, heading, description, project, validity, CTA, active/inactive, display order.

- Popup opens when the portal opens according to active/scheduled rules and has an X close action. Avoid repeatedly interrupting users if the configured display policy says first visit/session only.

19. Carousel CMS

- Dedicated AM Carousel Manager.
- Actions: upload, edit, delete/soft-delete, activate, deactivate, reorder, schedule, preview.
- Fields: banner image, title, subtitle, project, CTA, start date, end date, status, display order.
- Validate image type/size and dimensions; use consistent crop/aspect ratio.
- Render carousel responsively without layout shift.
- Reordering must be persisted transactionally and must not create duplicate display orders.
- Only active, scheduled and currently valid items appear to associates.

20. Reviews

- Associate creates review request with customer name, phone, email, project, booking, associate, date and interaction summary.
- System generates a unique review link.
- Customer submits ratings: overall experience, communication, property experience, associate support, plus written review.
- Submitted review is immutable: no edit action for Associate, AM or MD. If moderation is later required, it should be a separate moderation status/history, not mutation of the original review.
- Review analytics: average rating, total reviews, bookings, site visits, conversion and commission generated.

21. Notifications

- Categories: Booking, Project, Commission, Travel, Team, Approval, Offer, Site Visit, Review, System.
- Read/unread state.
- Direct navigation to the relevant record where safe.
- Use clear unread indicators and priority grouping.

22. Pending Requests

- Associate: own new-associate referral, booking verification, travel, site visit and other submissions.
- AM: associate referrals, booking verification, associate changes and operational requests.
- MD: new associates, team changes, travel, project approvals, commission changes and critical approvals.
- Approval/rejection actions must record actor, time, reason and before/after values where applicable.

23. My Profile & PWA Install

- Profile: photo, name, associate code, phone, WhatsApp, email, address, verification statuses, financial information as permitted, commission configuration, security.
- Sensitive PAN/Aadhaar values are masked.
- Security: change password, login/security information, logout, active device/session where applicable.
- Install App button appears only when installation is available or can be meaningfully guided.
- Use beforeinstallprompt where supported; otherwise provide platform/browser-specific instructions.
- Do not show Installed unless actual standalone/display-mode detection confirms installation.
- Include manifest, icons, theme color, standalone display and service worker.

24. FAQ & Tutorials

- FAQ categories: Account, Projects, Bookings, Commission, Team, Travel Allowance, Site Visits, Reviews, Offers, Security.
- FAQ search.
- Tutorials are a visual learning center, not plain text only.

- Tutorial topics: Create Booking, Refer Associate, How Commission Works, Schedule Site Visit, Request Travel Allowance.
- AM can manage tutorials/FAQ content if the final implementation includes the CMS controls specified.

25. Global Search

- Search Projects, Plots/Units, Bookings, Team Members, Offers and Site Visits.
- Search results must be permission-filtered server-side.
- Use debouncing, pagination and clear empty/error states.

26. Audit Trail

- Record who, what, when, before, after, IP/session/device for sensitive operational changes where appropriate.
- Examples: MD approved travel request; AM changed commission from 2% to 2.5%.
- Audit records are append-only for normal users and not editable through the portal.
- Never log passwords, JWTs or unnecessary sensitive payloads.

27. Recommended Technical Architecture

- Frontend: React.js + TypeScript + Vite.
- Routing: React Router.
- Server state: TanStack Query or equivalent; avoid putting server records into ad-hoc global state.
- Forms/validation: React Hook Form + Zod or equivalent.
- UI: reusable component system built on the supplied design tokens; professional SVG icon library.
- Backend: Node.js + TypeScript + Express or Fastify.
- API: REST with versioned /api/v1 routes.
- Database: PostgreSQL recommended for relational hierarchy, approvals, bookings, commission and auditability. Use Prisma or Drizzle ORM.
- Authentication: secure HttpOnly cookie JWT or short-lived access token + secure refresh/session strategy; choose one architecture and implement it consistently.
- Uploads: local/object storage abstraction; do not hard-code storage assumptions. Keep document/image URLs and metadata separate from business records.
- Testing: Vitest/Jest for unit tests, Supertest/API integration tests, Playwright for critical end-to-end workflows.
- Lint/format: ESLint + Prettier; TypeScript strict mode.
- Deployment: production React build + Node API, environment-based API URL, HTTPS, database migrations and seed for fixed MD/AM accounts.

28. Suggested Data Model

- User: id, role, status, email/phone/loginId, passwordHash, forcePasswordChange, createdAt, updatedAt.
- Session: id, userId, device metadata, token/session hash, createdAt, lastActiveAt, revokedAt; enforce one active session for MD/AM.
- AssociateProfile: userId, associateCode, personal/contact fields, sensitive identity/bank fields, verification flags, level, joiningDate, parentAssociateId, commission configuration.
- TeamRelationship: parentAssociateId, childAssociateId, status, effective dates; enforce no cycles.
- Project: id, name, location, type, status, verificationStatus, verifiedBy, verifiedAt, details.
- ProjectMedia: projectId, type, url, title, sortOrder.
- InventoryUnit: projectId, unit/plot number, type, size, price, status, bookingId.
- Booking: id, associateId, customer fields, projectId, inventoryUnitId, financial fields, status, verification metadata.
- CommissionPolicy/CommissionTransaction: project/associate scope, type, rate/fixed amount, transaction state, approval/audit fields.
- TravelRequest: associateId, travel data, documents, status, approval/payment fields.
- SiteVisit: customer, project, associate, schedule, status, outcome.

- Offer: type, projectId/associateId as applicable, content, validity, status, achievement configuration.
- CarouselItem: media, title/subtitle, projectId, CTA, validity, status, displayOrder.
- ReviewRequest/Review: unique token/link, customer data, booking/project/associate references, ratings, written review, immutable submittedAt.
- Notification: recipient, category, title/body, reference, readAt.
- ApprovalRequest: type, target, requester, reviewer, reason, status, rejectionReason, timestamps.
- FAQ/Tutorial: category, title, content/media, status, order.
- AuditLog: actor, action, entity, entityId, beforeJson, afterJson, metadata, createdAt.

29. API & Authorization Rules

- Every protected route derives current user identity from the server-side session/JWT, never from request body.
- Every resource query includes authorization scope before returning data.
- Use explicit policy/service functions such as canViewAssociate, canViewBooking, canModifyProject, canApproveRequest.
- Never expose a broad GET /associates/:id that returns all fields to all roles.
- Use DTOs/serializers to prevent accidental leakage of sensitive columns.
- Validate IDs and enum/status transitions server-side.
- Enforce inventory uniqueness/booking concurrency in the database transaction.
- Use idempotency or duplicate protection for sensitive submit actions where appropriate.

30. Phase-by-Phase Implementation Plan

- Phase 0 — Project audit & clean foundation: inspect folder, confirm React/Node split, establish TypeScript strictness, environment config, lint/test/build scripts, source control hygiene, no feature work until foundation is clean.
- Phase 1 — Foundation: monorepo or clean frontend/backend structure, database, migrations, API, auth, roles, fixed MD/AM accounts, password change, one-device sessions, basic shell, responsive tokens.
- Phase 2 — Associate management: creation, referral, generated code, AM/MD approval, profile, verification, hierarchy, add/remove requests.
- Phase 3 — Projects: CRUD, gallery, brochure, layout, inventory, offers, assignment, MD approval and verified badge.
- Phase 4 — Associate experience: dashboard, project browsing/details, active offers, notifications, quick actions, carousel and first-visit popup.
- Phase 5 — Bookings: create/update/verify/reject, inventory locking, statuses, My/Team bookings, hierarchy filtering.
- Phase 6 — Commission: policies, calculations, ledger, paid/pending/due, filters, approvals/audit.
- Phase 7 — Team: list/tree, downline visibility, stats and booking visibility.
- Phase 8 — Travel: request, documents, MD approval/rejection, payment/history.
- Phase 9 — Site visits: scheduling, statuses, outcomes, team visibility.
- Phase 10 — Offers + Carousel CMS: offer types, achievements, popup, carousel upload/reorder/schedule/preview.
- Phase 11 — Reviews: request, unique public link, customer form, immutable review, analytics.
- Phase 12 — Notifications: center, read/unread, categories, action links.
- Phase 13 — Help Center: FAQ search/categories, tutorials and content management.
- Phase 14 — Final UI/UX: skeletons, empty/error states, confirmations, animation, accessibility, mobile QA, performance and visual consistency.
- Phase 15 — Production hardening: security audit, permissions audit, route audit, concurrency tests, PWA install tests, build/deploy rehearsal, database backup/migration plan, final acceptance test.

31. Verification Gate After EVERY Phase

- Build status — pass.
- TypeScript/type-check — pass.

- Lint — pass.
- Unit tests — pass.
- Integration/API tests — pass.
- Runtime — pass.
- Console errors — none known.
- Routes — all relevant routes tested.
- Responsive — desktop/tablet/mobile checked.
- Authentication — checked where applicable.
- Role permissions — checked where applicable.
- Database/migrations — checked where applicable.
- Manual workflow — completed for changed feature.
- Regression check — previous phases still work.
- Known blocking errors — zero.
- Only then mark phase complete and continue.

32. Known Failure Patterns From the Previous Build — Must Not Repeat

These are implementation lessons carried into the new build from the earlier portal work. They are not additional business features; they are engineering constraints.

- Mobile sidebar strip/desktop sidebar leakage: do not leave desktop sidebar width, margin-left, fixed positioning or transform state active on mobile. Test at multiple widths and when opening/closing the mobile drawer.
- Mobile menu state: opening the menu must not restore or reveal the desktop shell. Drawer overlay, body scroll locking and main-content width must be coordinated.
- Poor alignment/UI density: build from a single tokenized spacing/grid system. Do a visual QA pass before adding more features.
- Missing install option: PWA installation must be implemented from the beginning, with real install detection and a Profile → Install App entry.
- Missing/unfinished profile: My Profile is a first-class module, not a placeholder.
- Carousel management/reordering: persist order server-side, validate order values, handle active/scheduled dates, preview, upload and responsive rendering.
- Popup/offer behavior: make the popup content CMS-controlled, responsive, closable and schedule-aware; do not hard-code campaign content.
- Authentication/cookie problems: when frontend and API are on different origins, explicitly configure CORS credentials, cookie SameSite/Secure/domain behavior, HTTPS and production environment variables. Test login, refresh, logout and protected requests in the deployed environment.
- Role leakage: never rely on hiding buttons. Test direct API access with manipulated IDs, URLs and payloads.
- Inventory race conditions: two users must not be able to confirm the same unit through simultaneous requests.
- Do not accumulate fixes by patching symptoms. When a test fails, identify the root cause and add a regression test.

33. Manual Terminal Test Routine

- Install dependencies cleanly.
- Run frontend type-check/build/lint/tests.
- Run backend type-check/build/lint/tests.
- Run database migration and seed checks.
- Start API and frontend in development mode.
- Use browser DevTools: Console, Network, Application/Storage and responsive device mode.
- Test login as MD, AM and Associate.

- Test forced password change for MD/AM.
- Test second-device rejection for MD/AM.
- Test direct unauthorized API requests by changing IDs.
- Test hierarchy visibility with at least A→B/C and B→D/E sample data.
- Test create → approve → verify → activate workflows.
- Test booking same inventory concurrently.
- Test mobile drawer/bottom navigation at narrow widths.
- Test PWA install and standalone mode.
- Run production build and serve the production artifacts locally before deployment.

34. Antigravity Working Contract

Antigravity must behave like a senior full-stack engineer and QA engineer, not a code generator that blindly adds files. At the beginning of each phase it must inspect the existing implementation, map the relevant files, identify dependencies and only then change code. After changes it must execute the verification gate, fix root causes and repeat the tests. It must stop at the current phase if a blocking defect remains.

- Never rewrite working modules unnecessarily.
- Never create duplicate components/routes/services to work around an existing defect.
- Prefer reusable components, service functions, validators and policies.
- Keep business logic out of presentation components where possible.
- Keep secrets in environment variables.
- Keep API contract types synchronized between frontend and backend.
- Add tests for every high-risk authorization, approval, booking and inventory rule.
- Before changing authentication or responsive shell, inspect existing behavior and test regression paths.
- Report changed files and test results after each phase.

35. Master Antigravity Prompt

The following prompt is intended to be pasted as the single master instruction before starting the new project. It deliberately combines product scope, architecture, UI, security, workflow and QA rules so the implementation does not drift.

MASTER ANTIGRAVITY PROMPT — COPY AS ONE BLOCK

```
You are the lead engineer, product architect, UI engineer and QA engineer for the Sonthillu Constructions Associate Partner Portal.

PROJECT:
Build a completely standalone Associate Partner Portal as ONE responsive web application. Frontend MUST be React.js + TypeScript. Backend MUST be Node.js + TypeScript. Use a relational database such as PostgreSQL. The app must work in desktop/mobile browsers and be installable as a real PWA. The final deployment target is Hostinger.

ABSOLUTE SCOPE BOUNDARY:
This portal is independent. DO NOT integrate with CRM, EMS, Customer Portal, existing CRM/EMS databases, or unrequested external business APIs. It has its own authentication, authorization, database, workflows, projects, bookings, commissions, teams, offers, carousel, site visits, travel, reviews, notifications, FAQ and tutorials.

ROLES:
Exactly three primary roles:
1. MD – highest authority.
2. Associate Manager (AM) – operational administrator.
3. Associate – partner/sales user.

SOURCE DOCUMENTS:
Use the supplied Associate Partner Portal Master Development Instructions, associate portal app specification, Sonthillu Constructions Master UI Design System and exact Sonthillu logo as the source of truth. Do not invent conflicting design or business rules.

TECHNICAL BASELINE:
- React + TypeScript + Vite.
- React Router.
- TanStack Query or equivalent for server state.
- React Hook Form + Zod or equivalent for forms/validation.
- Node.js + TypeScript + Express/Fastify.
- REST API under /api/v1.
- PostgreSQL + Prisma/Drizzle.
```

- Strict TypeScript.
- ESLint + Prettier.
- Unit/integration/E2E tests.
- Secure HttpOnly-cookie authentication or another explicitly chosen secure session architecture; configure cross-origin behavior correctly if frontend and API are on different origins.
- Environment variables for all secrets and deployment URLs.

UI:

Follow Sonthillu design system exactly:

Primary Navy #203B73; Brand Gold #E0B040; App Background #F4FAFC; Deep Navy #172A52; Action Blue #4268E8; Card Surface #EAF3F8; Primary Text #172033; Muted Text #667085; Border #D7E3EA.

Use Inter/Manrope/Plus Jakarta Sans or equivalent.

Use 8px-based spacing rhythm, 14-20px card radii, 44px minimum controls, pale blue canvas, white/soft cards, restrained shadows, no heavy gradients, no neon/glow, no emoji as primary navigation icons.

Desktop: 240-260px sidebar + top bar + 12-column content grid.

Tablet: collapsed sidebar + 8-column grid.

Mobile: header + content + bottom navigation + drawer for secondary navigation + 4-column grid.

Never allow desktop sidebar dimensions to leak into mobile. Test drawer open/close and body scroll behavior.

NAVIGATION:

Dashboard, Projects, Bookings, Commissions, Team, Offers, Pending Requests, Travel Allowance, Site Visits, Notifications, Reviews, My Profile, FAQ, Tutorials.

AUTH & SECURITY:

MD and AM are fixed accounts with temporary passwords. First login forces password change. Temporary password becomes invalid after successful change. MD

and AM allow only one active device/session; second login is rejected with a clear message. Profile supports session/device information and revoke/sign-out. Associates may use multi-device access unless later changed. Enforce authorization on the server/data-access layer. Never trust user IDs, role, parent IDs or visibility scopes from the client. Prevent IDOR by direct URL/API manipulation. Protect Aadhaar, PAN, bank account, IFSC, emergency contact and private address. Use DTOs/serializers that exclude sensitive fields. Add validation, rate limiting, security headers and appropriate CSRF protection for cookie-auth architecture.

HIERARCHY:

Downward-only visibility. Example A→B/C and B→D/E means A sees A/B/C/D/E; B sees B/D/E; B cannot see A/C. Enforce no cycles. Apply the same scope to bookings and team records.

MODULES:

MD Dashboard and approvals; Associate Management; Projects; Associate Dashboard; Bookings; Commission; Team; Travel Allowance; Site Visits; Offers; Carousel CMS; Reviews; Notifications; Pending Requests; My Profile; FAQ; Tutorials; Global Search; Audit Trail; PWA install.

PROJECTS:

Cards show project name, verified badge, location, type, status, brochure, layout, gallery, available inventory, details, associate offer and customer offer. Detail tabs: Overview, Gallery, Layout, Brochure, Available Units, Offers. Associates see only available inventory. MD approval is the ONLY trigger for Verified Project badge.

BOOKINGS:

Capture customer details, project, unit/plot, property type, size, price, booking date, expected amount, payment mode, booking amount, notes and documents. Workflow SUBMITTED → AM verification/UNDER REVIEW → VERIFIED → CONFIRMED, with rejected/payment-pending/cancelled states. Prevent unavailable inventory and concurrent double booking at database transaction level. After verified booking, inventory becomes unavailable.

COMMISSION:

Support percentage and fixed commission. Dashboard total/received/pending. Ledger and date/project filters. Financial changes must be audited and policy changes follow MD final authority.

TEAM:

List and tree views. Downline visibility. Only show allowed team fields to associates; never leak phone/email/address/bank/Aadhaar/PAN/emergency data.

TRAVEL:

Request form with date, from/to, purpose, project, customer, distance, mode, amount, bill and notes. Workflow Pending → MD Review → Approved/Rejected → Paid. Preserve history.

SITE VISITS:

My Site Visits and Team Site Visits. Schedule, On the Way, Arrived, Customer Met, Completed, Cancelled, Rescheduled. Capture outcome, interest, property viewed and follow-up.

OFFERS:

Separate Associate Offers, Customer Offers, Project Offers and Achieved Offers. Support progress/achievement UI.

CAROUSEL:

AM-only CMS. Upload/edit/soft-delete, activate/deactivate, reorder, schedule, preview. Fields banner image, title, subtitle, project, CTA, start/end dates, status, display order. Persist ordering server-side. Validate media. Render responsively.

FIRST-VISIT OFFER POPUP:

AM controls image, heading, description, project, validity, CTA, active/inactive and display order. Show according to active/schedule rules when portal opens. Include close button. Make it responsive and avoid repeated intrusive display beyond configured behavior.

REVIEWS:

Associate creates review request; system generates unique review link; customer submits ratings for overall experience, communication, property experience, associate support and written review. Submitted reviews are immutable. No edit button for Associate/AM/MD. Analytics for ratings, reviews, bookings, visits, conversion and commission.

NOTIFICATIONS:

Booking, Project, Commission, Travel, Team, Approval, Offer, Site Visit, Review, System. Read/unread and contextual links.

PROFILE/PWA:

Profile includes identity, associate code, contact, address, verification, financial/security sections. Mask sensitive values. Include Change Password, security information, logout and Install App. Implement real PWA manifest, icons, standalone display, theme color, service worker and installability. Detect installed state correctly; never fake it. Browser mode and installed PWA use the same app/data.

FAQ/TUTORIALS:

FAQ categories and search. Tutorials are visual learning content with video/tutorial management.

GLOBAL SEARCH:

Search project, plot/unit, booking, team member, offer and site visit, with server-side permission filtering.

AUDIT:

Append-only audit trail: who, what, when, before, after, IP/session/device where appropriate. Never log passwords/tokens.

DATA INTEGRITY:

Do not permanently delete important records. Prefer statuses and soft deletion. Preserve bookings, commissions, reviews, approvals, travel claims, projects and team history.

IMPLEMENTATION ORDER:

Phase 0 foundation audit.
Phase 1 foundation/auth/security/shell/PWA base.
Phase 2 associate management/hierarchy.
Phase 3 projects/inventory.
Phase 4 associate dashboard/carousel/popup.
Phase 5 bookings.
Phase 6 commission.
Phase 7 team.
Phase 8 travel.
Phase 9 site visits.
Phase 10 offers/carousel CMS.
Phase 11 reviews.
Phase 12 notifications.
Phase 13 FAQ/tutorials.
Phase 14 UI/UX/accessibility/performance.
Phase 15 production hardening/deployment.

PHASE GATE:

At every phase:

1. Inspect current project first.
2. Implement only current phase.
3. Run app.
4. Run build.
5. Run TypeScript check.
6. Run lint.
7. Run unit/integration tests.
8. Test changed workflows.
9. Check console/runtime errors.
10. Check all relevant routes.
11. Check desktop/tablet/mobile.
12. Check auth/permissions/database where applicable.
13. Fix root causes.
14. Repeat full phase tests.
15. Only mark complete when there are zero known blocking errors.
16. Do not proceed automatically to the next phase; wait for the next phase instruction.

DO NOT REPEAT THESE PREVIOUS FAILURE MODES:

- No mobile desktop-sidebar strip.
- No mobile drawer that leaves desktop layout offsets active.
- No poor alignment or inconsistent spacing.

- No missing Profile or Install App.
- No fragile carousel reorder implementation.
- No hard-coded popup content.
- No cross-origin JWT/cookie configuration guesswork.
- No frontend-only authorization.
- No inventory double-booking.
- No symptom-patching without regression tests.

WHEN YOU START:

First inspect the entire project tree and existing package/config files. Do not generate a replacement project blindly. Produce a concise architecture audit, identify conflicts and obsolete files, then implement Phase 0 only. Preserve useful existing work unless it conflicts with this specification. Do not create duplicate routes/components/services.

OUTPUT AFTER EACH PHASE:

PHASE STATUS

- Implemented:
- Tested:
- Errors found:
- Errors fixed:
- Final test result:
- Files/modules changed:
- Remaining known issues:
- Exact commands executed:

36. Phase Prompts

Use these as short follow-up prompts after the master prompt. They are intentionally focused so Antigravity does not consume its context repeatedly rebuilding unrelated modules.

PHASE 0 — AUDIT

Inspect the entire current project. Do not implement features yet. Identify frontend/backend structure, package managers, build scripts, database, auth, routes, duplicated files, broken modules, current responsive shell and existing tests. Compare against the master specification. Establish a clean baseline. Run build/type-check/lint/tests and report every failure. Fix only foundation/configuration defects needed for a reliable baseline. Stop after the verification gate.

PHASE 1 — FOUNDATION

Implement the React + TypeScript frontend shell, Node + TypeScript API, PostgreSQL schema/migrations, secure authentication, MD/AM fixed accounts, Associate account model, forced first-login password change, one-device MD/AM session rule, secure logout/session revoke, role authorization, responsive desktop/tablet/mobile shell, design tokens, PWA manifest/service worker/install foundation. Test direct unauthorized API access. Pay special attention to cross-origin cookie/CORS configuration if applicable. Stop after all tests pass.

PHASE 2 — ASSOCIATES

Implement Associate Management: AM create associate, generated permanent code, associate referral, AM review, MD approval, profile, verification, parent/team hierarchy, add/remove team requests. Enforce downward visibility and no-cycle hierarchy server-side. Test IDOR and sensitive-data leakage. Stop after verification gate.

PHASE 3 — PROJECTS

Implement projects, media/gallery, brochure, layout, inventory units, project offers, associate offers, project assignment, project approval and MD Verified badge. Only MD approval can set verified status. Associates see only available inventory. Add authorization and inventory tests. Stop after verification gate.

PHASE 4 — ASSOCIATE EXPERIENCE

Implement Associate Dashboard, project browsing/details, active offers, upcoming site visits placeholders only if data exists, notifications entry, quick actions, carousel rendering and the AM-controlled first-visit promotional popup. Fix responsive behavior before adding visual polish. Test mobile drawer/sidebar thoroughly. Stop after verification gate.

PHASE 5 — BOOKINGS

Implement booking creation/update, customer details, project/unit selection, document handling, AM verification, statuses, My Bookings, permitted Team Bookings, hierarchy filtering, duplicate/conflict validation and transactional inventory locking. Test concurrent booking attempts. Stop after verification gate.

PHASE 6 — COMMISSION

Implement commission configuration for percentage/fixed amount, approval boundary, commission calculations, ledger, received/pending/due states, date/project filters and audit logging. Test financial calculations and authorization. Stop after verification gate.

PHASE 7 — TEAM

Implement team list/tree views, downline visibility, team statistics, booking visibility and team request states. Test parent/sibling/child access by API, not only UI. Stop after verification gate.

PHASE 8 — TRAVEL

Implement travel allowance request form, supporting documents, MD approval/rejection, payment update and history/filtering. Preserve all records and rejection reasons. Stop after verification gate.

PHASE 9 — SITE VISITS

Implement site visit scheduling, customer/project data, statuses, My/Team views, visit updates and outcome/follow-up fields. Test mobile usability. Stop after verification gate.

PHASE 10 — OFFERS + CAROUSEL CMS

Implement Associate/Customer/Project/Achieved offers, achievement progress, Carousel Manager upload/edit/activate/deactivate/reorder/schedule/preview, server-persisted display order and the first-visit popup CMS controls. Test media validation, scheduling and responsive rendering. Stop after verification gate.

PHASE 11 — REVIEWS

Implement review request creation, unique review link/token, public customer review page, rating categories, written review, immutable submission and review analytics. Test that submitted reviews cannot be edited by any role. Stop after verification gate.

PHASE 12 — NOTIFICATIONS

Implement notification center, categories, read/unread, approval/booking/commission/travel/site-visit/offer/review/system notifications and permission-safe deep links. Stop after verification gate.

PHASE 13 — HELP

Implement searchable FAQ categories and visual tutorial center, plus AM content management if included in the current scope. Stop after verification gate.

PHASE 14 — FINAL UX

Do not add new business features. Perform visual QA across desktop/tablet/mobile, alignment, spacing, typography, overflow, sidebar/drawer behavior, bottom navigation, skeletons, empty/error states, confirmation dialogs, loading states, accessibility, keyboard focus, contrast, image ratios, motion and performance. Fix defects and add regression tests. Stop after verification gate.

PHASE 15 — PRODUCTION

Perform full production hardening: security review, authorization matrix tests, database migration rehearsal, backup/restore plan, environment validation, PWA install/standalone tests, production frontend build, Node API production start, HTTPS/cookie/CORS validation, logging, health endpoint, route audit and end-to-end smoke test. Deploy only after the production checklist passes.

37. Approval Matrix

- View projects: Associate ✓ / AM ✓ / MD ✓
- Create project: Associate — / AM ✓ / MD ✓
- Approve project: Associate — / AM — / MD ✓
- Create associate: Associate — / AM ✓ / MD ✓
- Refer associate: Associate ✓ / AM ✓ / MD ✓
- Approve new associate: Associate — / AM — / MD ✓
- Modify associate: Associate — / AM ✓ / MD ✓
- Request/Process team changes: Associate requests / AM processes / MD approves where required.
- Configure commission: Associate view / AM configure / MD final authority.
- Request travel: Associate ✓ / AM view/process / MD approve.
- Approve travel: Associate — / AM — / MD ✓.
- Create booking: Associate ✓ / AM verify / MD view.
- Verify booking: Associate — / AM ✓ / MD view.
- Manage carousel: Associate — / AM ✓ / MD view.
- Manage offers: Associate — / AM ✓ / MD view; MD approval optional where business policy requires.
- View team: Associate downline / AM all / MD all.
- Sensitive data: Associate own limited / AM according to operational permission / MD according to authority.
- Manage tutorials/FAQ: Associate — / AM ✓ / MD ✓.

38. Production Deployment Checklist

- Create production database and run migrations.
- Seed fixed MD/AM accounts securely; change temporary passwords immediately.
- Set frontend API base URL to production API.

- Enable HTTPS everywhere.
- Configure CORS exact allowed frontend origin(s), credentials and methods.
- Configure cookie attributes appropriate to the actual frontend/API origin relationship.
- Set secure secrets in environment variables; never commit them.
- Configure upload/storage limits and allowed MIME types.
- Build React frontend and verify asset paths for Hostinger.
- Deploy Node API and verify health endpoint.
- Run login/password-change/logout smoke tests.
- Run MD/AM one-device session test.
- Run Associate hierarchy/IDOR tests.
- Run project/inventory/booking tests.
- Run carousel/popup tests.
- Run PWA install tests on supported desktop/mobile browsers.
- Check browser console and network for CORS/401/403/404/500 errors.
- Verify responsive layout on actual mobile browser.
- Configure database backups and migration rollback procedure.
- Do not declare production ready until the complete smoke test passes.

Source Documents & Evidence

This blueprint was synthesized from the three supplied PDFs and the exact supplied logo asset. The first portal instruction document establishes independence, the three-role model, phase gates, security and PWA requirements. The portal specification establishes the module/workflow scope, hierarchy, carousel, popup, reviews, approval matrix and implementation phases. The UI design system establishes the visual palette, typography, spacing, responsive behavior and component rules.